

ECE 30: Introduction to Computer Engineering
Recursive Blocked Matrix Multiplication for ML Inference
Programming Project: Spring 2026
Due: Thursday, June 4th, 2026 at 11:00 AM

1 Introduction

In this project, you will implement a recursive blocked matrix multiplication algorithm in LEGv8 assembly to compute the product of two square matrices. Matrix multiplication is one of the most important computations in computer engineering. It appears in graphics, scientific computing, signal processing, and, most importantly for this project, machine learning inference. In many machine learning models, most of the execution time is spent performing repeated matrix multiplications between an input matrix and a weight matrix. Because of this, efficient matrix multiplication is one of the most important topics in high-performance computing.

A straightforward way to multiply two matrices is to use three nested loops. For example, to compute $C = A \times B$, one may iterate through each row of A , each column of B , and accumulate the dot product. Although this method is simple and mathematically correct, it is often not the best way to use the memory hierarchy of a real machine. Modern processors can perform arithmetic operations much faster than they can fetch data from main memory. As a result, a large part of high-performance algorithm design is not just about reducing arithmetic, but also about organizing memory accesses to improve locality and reuse.

A common technique for improving matrix multiplication is called **blocking**. Instead of multiplying the entire matrices at once, we divide them into smaller submatrices, also called blocks or tiles, and multiply those smaller blocks. This allows a processor to reuse data that has already been loaded into cache, reducing the cost of memory access. In high-performance software libraries, matrix multiplication is typically organized into two levels:

- a high-level decomposition that divides the problem into smaller subproblems, and
- a low-level kernel that computes multiplication efficiently for small blocks.

In this project, you will implement a simplified version of this idea using recursion. Instead of only using iterative loops, your code will recursively split a matrix into four quadrants and compute the output block by block. If the current matrix block is small enough, your code will stop recursing and call a direct multiply-accumulate routine.

In addition to computing the matrix product, the program also performs a **trace calculation**. The trace of a matrix is defined as the sum of its diagonal elements. In this project, each base-case multiplication contributes the trace of its resulting submatrix. These partial traces are accumulated through the recursive calls and returned as a single scalar value, which can help you verify correctness without inspecting the entire output matrix.

1.1 Method Overview

We compute:

$$C = A \times B$$

by dividing each matrix into four quadrants:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

Then the result is:

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

where each block is computed as follows:

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}$$

$$C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

$$C_{21} = A_{21}B_{11} + A_{22}B_{21}$$

$$C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

The algorithm recursively applies this decomposition until the submatrix size reaches a predefined **base** size. At that point, a standard triple-loop multiplication is used.

For the trace calculation, only the diagonal blocks (C_{11} and C_{22} at each level) contribute to the trace. When a base case is reached, the trace of that submatrix is computed directly and returned. These values are then accumulated through recursive calls to produce the final trace.

1.2 4×4 Example (Base = 2)

We can illustrate the method with a concrete example:

Consider multiplying two 4×4 matrices with **base** = 2. Each matrix is split into four 2×2 blocks.

Let:

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

Split into quadrants:

$$A_{11} = \begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix}, \quad A_{12} = \begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix}, \quad A_{21} = \begin{bmatrix} 9 & 10 \\ 13 & 14 \end{bmatrix}, \quad A_{22} = \begin{bmatrix} 11 & 12 \\ 15 & 16 \end{bmatrix}$$

$$B_{11} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad B_{12} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad B_{21} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad B_{22} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

We now compute each submatrix explicitly.

Step 1: C_{11}

$$A_{11}B_{11} = \begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 5 & 6 \end{bmatrix}$$

$$A_{12}B_{21} = \begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix}$$

$$C_{11} = \begin{bmatrix} 4 & 6 \\ 12 & 14 \end{bmatrix}$$

Step 2: C_{12}

$$A_{11}B_{12} = \begin{bmatrix} 2 & 1 \\ 6 & 5 \end{bmatrix}, \quad A_{12}B_{22} = \begin{bmatrix} 3 & 4 \\ 7 & 8 \end{bmatrix}$$

$$C_{12} = \begin{bmatrix} 5 & 5 \\ 13 & 13 \end{bmatrix}$$

Step 3: C_{21}

$$A_{21}B_{11} = \begin{bmatrix} 9 & 10 \\ 13 & 14 \end{bmatrix}, \quad A_{22}B_{21} = \begin{bmatrix} 11 & 12 \\ 15 & 16 \end{bmatrix}$$

$$C_{21} = \begin{bmatrix} 20 & 22 \\ 28 & 30 \end{bmatrix}$$

Step 4: C_{22}

$$A_{21}B_{12} = \begin{bmatrix} 10 & 9 \\ 14 & 13 \end{bmatrix}, \quad A_{22}B_{22} = \begin{bmatrix} 11 & 12 \\ 15 & 16 \end{bmatrix}$$

$$C_{22} = \begin{bmatrix} 21 & 21 \\ 29 & 29 \end{bmatrix}$$

Final Result

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} 4 & 6 & 5 & 5 \\ 12 & 14 & 13 & 13 \\ 20 & 22 & 21 & 21 \\ 28 & 30 & 29 & 29 \end{bmatrix}$$

Trace Calculation The trace is the sum of diagonal elements:

$$\text{trace}(C) = 4 + 14 + 21 + 29 = 68$$

In the recursive algorithm, this value is obtained by summing the traces of the diagonal blocks:

$$\begin{aligned}\text{trace}(C) &= \text{trace}(C_{11}) + \text{trace}(C_{22}) \\ &= (4 + 14) + (21 + 29) = 18 + 50 = 68\end{aligned}$$

2 Implementation

In this project, you will need to write four functions in LegV8, namely *getAddr*, *base-MultiplyAdd*, *splitOffset*, and *recBlockMul*. A top-level driver function *runInference* is provided to prepare the data and call these functions.

2.1 Important Notes (Please Read Before Starting)

2.1.1 Matrix Size Assumptions

In this project, you may assume that **all matrices are square**, and their dimensions are powers of 2 and the base block size is a power of 2. This allows every recursive step to split the current matrix evenly into **four equal-sized submatrices** and guarantees that the recursive decomposition continues cleanly until the block size reaches the given base case.

2.1.2 Matrix Storage & Indexing

Matrices are stored as 1D arrays in **row-major order**. For example, the matrix

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

is stored in memory as:

$$[1, 2, 3, 4]$$

You should also index the row and column of a matrix **starting from 0**. In the above example, element 1 is at row 0, column 0.

2.1.3 Procedure Interface & Call Convention

Follow the **Procedure Call Convention** below for calling procedures, passing registers, and managing the stack. Procedures should not make assumptions about the implementation of others. Code that violates these conventions will be penalized.

Name	Number	Usage	Preserved (Pt)
–	0–7	Arguments 1–8 / Return values	No
–	8	Indirect result location (used to return pointers)	No
–	9–15	Temporary (saved by caller if necessary)	No
–	16–18	Reserved (or temporary)	No
–	19–26	Saved registers (by callee)	Yes
GP	27	Global pointer	Yes
SP	28	Stack pointer	Yes
FP	29	Frame pointer	Yes
LR	30	Return address	Yes
XZR	31	Constant 0	–

Table 1: LEGv8 Register Usage and Calling Convention

2.1.4 Code Documentation

We expect your code to be well-commented. Each instruction should have a meaningful description of the operation.

A poor comment:

```
// X1 is X2 + X3  
ADD X1, X2, X3
```

A better comment:

```
// Compute row-major linear index for the current matrix element  
ADD X1, X2, X3
```

Note: Your comments will not be graded, but your TA may refuse to proofread your code if your code is not well-commented.

2.1.5 Interoperability.

Your project **must use the provided template file**. While you are free to add additional helper functions, your code must implement the following functions according to the specification given below.

The grader should be able to remove any of these functions (and any non-essential helper functions it may use) from your code and use them in another person's programming assignment without issue. In other words, do not change the name, input registers, and output registers of the functions.

2.2 Top Level Driver: runInference(A, B, C, n, base)

This function is the top-level driver that calls `recBlockMul` to compute the product and prints the results. You may think of this as one simple machine learning inference step, where matrix A is a weight matrix, matrix B is an input matrix, and matrix C is the output.

The function is already implemented in the provided template. The `PRINTMATRIX` function used for displaying results is also already implemented.

2.2.1 C code

```
1 // Setup code before runInference (in assembly):
2 // X0 = address of matrix A (loaded from data section)
3 // X1 = address of matrix B (loaded from data section)
4 // X2 = address of matrix C (loaded from data section)
5 // X3 = n (matrix dimension, e.g., 4 for 4x4)
6 // X4 = base (base block size, e.g., 2)
7 // X5 = stride (matrix stride, e.g., 4)
8
9 void runInference(long* A, long* B, long* C, long n, long base, long stride)
10 {
11     // Call recursive blocked multiplication
12     long trace = recBlockMul(A, B, C, n, base, stride); // trace returned
13     // Print trace (newline, trace value, newline)
14     putchar('\n'); // PUTCHAR with newline character
15     putint(trace); // PUTINT with trace value
16     putchar('\n'); // PUTCHAR with newline
17
18     // Print result matrix C for visualization
19     PRINTMATRIX(C, n, stride);
20 }
```

Note: The input matrices A , B , C and arguments n , $base$, $stride$ are pre-initialized in the data section before the function call. You can check the provided data file for details on how to set up these values.

2.2.2 Example

We use the `testData_4by4_base2.txt` file for this example. Before starting:

- A matrix: {1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16}
- B matrix: {16,15,14,13,12,11,10,9,8,7,6,5,4,3,2,1}
- C matrix: {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0}
- stride: 4

- n: 4
- base: 2

After you finish your implementation and run the program on the simulator, it should output:

```
996
80 70 60 50
240 214 188 162
400 358 316 274
560 502 444 386
```

Where 996 is the trace of the resulting matrix C .

And the resulting matrix C is:

$$C = \begin{bmatrix} 80 & 70 & 60 & 50 \\ 240 & 214 & 188 & 162 \\ 400 & 358 & 316 & 274 \\ 560 & 502 & 444 & 386 \end{bmatrix}$$

2.3 Function 1: getAddr(base, row, col, stride)

This function computes the address of an element in a matrix stored in row-major order. Given the base address of the matrix, the row index, the column index, and the row stride, it calculates the address of the element at (row, col) and returns it.

2.3.1 C code

```
1 long* getAddr(long* base, long row, long col, long stride) {  
2     // Calculate the address of the element at (row, col).  
3     // row and col are 0-indexed.  
4     return base + row * stride + col;  
5 }
```

Note: The offset calculation in C uses pointer arithmetic (in units of `long`), while in LEGv8 assembly addresses are in bytes. You will need to account for this difference when computing addresses.

2.3.2 Parameters

- X5: The address of (pointer to) the matrix base.
- X6: The row index.
- X7: The column index.
- X8: The row stride of the matrix.

2.3.3 Return Value

- The returned address should be returned in X5.

2.3.4 Example

Before starting:

- matrix = {1,2,3,4}

Input:

- X5: pointer to matrix
- X6: 0
- X7: 1
- X8: 3

Returns:

- X5: pointer to the element at row 0, column 1, which is the address of element 2

2.4 Function 2: baseMultiplyAdd(A, B, C, n, stride)

This function performs direct multiplication for a small matrix block and accumulates the result into C . It also computes and returns the trace of the resulting $n \times n$ block of C .

Note: You'll need to call `getAddr` to access and modify elements in the matrix blocks in your implementation.

2.4.1 C code

```
1 long baseMultiplyAdd(long* A, long* B, long* C, long n, long stride) {
2     long t = 0; // Accumulate trace of this n x n block of C
3
4     for (long i = 0; i < n; i++) { // Iterate over rows
5         for (long j = 0; j < n; j++) { // Iterate over columns
6             long sum = 0; // Dot product for C[i][j]
7
8             for (long k = 0; k < n; k++) { // Compute A row i * B column j
9                 sum += A[i * stride + k] * B[k * stride + j]; // A[i][k] * B
10                [k][j]
11            }
12            C[i * stride + j] += sum; // Accumulate into C[i][j]
13        }
14
15        t += C[i * stride + i]; // Add diagonal element to trace
16    }
17
18    return t; // Return trace of this block
19 }
```

2.4.2 Parameters

- X0: The pointer of matrix block A .
- X1: The pointer of matrix block B .
- X2: The pointer of matrix block C .
- X3: The block size n .
- X4: The row stride of the matrices.

2.4.3 Return Value

- X0: The trace of the resulting $n \times n$ block of C .
- The block C should be updated in memory.

2.4.4 Example

This example shows multiplies a 2×2 block inside a larger 4×4 matrix.

Before starting:

- A block starts at the top-left of:

$$A = \begin{bmatrix} 1 & 2 & 0 & 0 \\ 3 & 4 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

- B block starts at the top-left of:

$$B = \begin{bmatrix} 5 & 6 & 0 & 0 \\ 7 & 8 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

- C block starts at the top-left of a zero matrix.

- $n = 2$

- `stride = 4`

Input:

- X0: pointer to the start of the A block
- X1: pointer to the start of the B block
- X2: pointer to the start of the C block
- X3: 2
- X4: 4

Output:

- The updated C matrix is:

$$C = \begin{bmatrix} 19 & 22 & 0 & 0 \\ 43 & 50 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

- X0: 69 (trace = 19 + 50)

2.5 Function 3: splitOffset(base, n, quadrant, stride)

This function returns the base address of a submatrix quadrant.

The quadrant numbering is:

- 0: top-left
- 1: top-right
- 2: bottom-left
- 3: bottom-right

2.5.1 C code

```
1 long* splitOffset(long* base, long n, long quadrant, long stride) {
2     long half = n / 2;
3     if (quadrant == 0) return base;                // top-left
4     if (quadrant == 1) return base + half;         // top-right
5     if (quadrant == 2) return base + half * stride; // bottom-left
6     return base + half * stride + half;           // bottom-right
7 }
```

2.5.2 Parameters

- X0: The address of (pointer to) the matrix base.
- X1: The current block size n .
- X2: Quadrant number.
- X3: The row stride.

Note: ‘stride’ is the stride of the parent matrix, not the submatrix. This allows quadrants to correctly reference the original memory layout.

2.5.3 Return Value

- Returns the address of the selected quadrant in X8.

2.5.4 Example

Before starting:

- Matrix:

$$\begin{bmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 2 & 2 & 3 & 3 \\ 2 & 2 & 3 & 3 \end{bmatrix}$$

- Flattened form: $\{0,0,1,1, 0,0,1,1, 2,2,3,3, 2,2,3,3\}$

Input:

- X0: pointer to matrix
- X1: 4
- X2: 3
- X3: 4

Returns:

- X8: pointer to the bottom-right quadrant (the 2x2 block of 3's)

2.6 Function 4: recBlockMul(A, B, C, n, base, stride)

This function performs recursive blocked multiplication. If the current block size is small enough, it calls `baseMultiplyAdd`. Otherwise, it splits the matrices into quadrants and recursively computes the corresponding block products. The return value accumulates the sum of traces of all diagonal base-case blocks.

2.6.1 C code

```
1 long recBlockMul(long* A, long* B, long* C, long n, long base, long stride)
2 {
3     if (n <= base) {
4         return baseMultiplyAdd(A, B, C, n, stride);
5     }
6
7     long* A11 = splitOffset(A, n, 0, stride);
8     long* A12 = splitOffset(A, n, 1, stride);
9     long* A21 = splitOffset(A, n, 2, stride);
10    long* A22 = splitOffset(A, n, 3, stride);
11
12    long* B11 = splitOffset(B, n, 0, stride);
13    long* B12 = splitOffset(B, n, 1, stride);
14    long* B21 = splitOffset(B, n, 2, stride);
15    long* B22 = splitOffset(B, n, 3, stride);
16
17    long* C11 = splitOffset(C, n, 0, stride);
18    long* C12 = splitOffset(C, n, 1, stride);
19    long* C21 = splitOffset(C, n, 2, stride);
20    long* C22 = splitOffset(C, n, 3, stride);
21
22    long t = 0;
23
24    // C11 = A11B11 + A12B21 (diagonal block)
25    recBlockMul(A11, B11, C11, n/2, base, stride);
26    t += recBlockMul(A12, B21, C11, n/2, base, stride);
27
28    // C12 = A11B12 + A12B22 (off-diagonal block)
29    recBlockMul(A11, B12, C12, n/2, base, stride);
30    recBlockMul(A12, B22, C12, n/2, base, stride);
31
32    // C21 = A21B11 + A22B21 (off-diagonal block)
33    recBlockMul(A21, B11, C21, n/2, base, stride);
34    recBlockMul(A22, B21, C21, n/2, base, stride);
35
36    // C22 = A21B12 + A22B22 (diagonal block)
37    recBlockMul(A21, B12, C22, n/2, base, stride);
38    t += recBlockMul(A22, B22, C22, n/2, base, stride);
39
40    return t;
41 }
```

Hint: Your assembly implementation need not follow the exact order as shown in the C code. You can compute the pointer and blocks in any order, as long as you correctly accumulate the trace from the diagonal blocks and produce the correct final output matrix C .

2.6.2 Parameters

- X0: Pointer to matrix block A .
- X1: Pointer to matrix block B .
- X2: Pointer to matrix block C .
- X3: Current block size n .
- X4: Base block size.
- X5: Row stride of the matrices.

2.6.3 Return Value

- X0: The sum of traces of all diagonal base-case blocks.

2.6.4 Example

Before starting:

- A matrix: {1,2,0,1,0,1,3,2,2,0,1,1,1,1,0,2}
- B matrix: {1,0,2,1,3,1,0,2,1,4,2,0,0,1,1,3}
- C matrix: {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0}
- n: 4
- base: 2
- stride: 4

When exiting:

- C matrix: {7,3,3,8,6,15,8,8,3,5,7,5,4,3,4,9}
- X0: 68 (sum of traces from diagonal blocks)

3 Instructions

The project is due before the Thursday, June 4th lecture (Week 10) at 11:00 AM. Any questions asked about the project after Tuesday, June 2nd, will not be answered.

You are provided with the `starter.s` file and three test files named `testData_XXbyXX_baseY.txt` to test your code. You need to understand the `runInference` function and how tests work.

Try to test the behavior of each function independently, rather than trying to code all of them at once. A good way to test your code is to modify the main function and test functions one by one. It will make later debugging far easier. Also, we will test functions individually, across matrices of varying sizes.

As mentioned in the instructions section, you should not submit work from others. The code submitted should be your work. This means:

- Do not copy the code from others.
- Do not hire others to write the project for you.
- Do not use generative AI such as ChatGPT to generate your project code.

We will be checking through your code. As LEGv8 is not a commonly used assembly language, generative AI will make many mistakes, and it is very easy to tell if a given code is written by a human or by a generative AI. Any detected violation will be reported to the AI office.

4 Grading

- `getAddr` (1 point): 2 test cases, 0.5 point each
- `baseMultiplyAdd` (3 points): 3 test cases, 1 point each
- `splitOffset` (2 points): 2 test cases, 1 point each
- `recBlockMul` (4 points): 4 test cases, 1 point each
- Assembly flow (5 points): Your code should compile, run, or terminate correctly. And your code should follow the calling convention or work with our solution code.

Note: Your comments will not be graded, but your TA may refuse to proofread your code if your code is not well-commented.

5 Miscellaneous (please read)

- You should show up in YOUR week 5 lab session. Otherwise, your partner has the right to request a new partner. If you cannot make it, notify ecethirty@gmail.com in advance.
- Week 7 lab sessions will also be utilized for the project.
- Project office hour: 3-5 pm Monday at EBU1 5101B.
- If you get stuck on any function, reviewing the recursion lectures might help, especially the episodes covering recursive procedure calls and stack usage.
- If encountering a syntactic error, Legiss will report the instruction that caused the error at the bottom of the left panel (see also the bottom-right panel). This information is often helpful.
- There is a BUG in LEGv8: You cannot have comments right after the instruction. You must use a SPACE to separate them!

```
LDUR X11, [X0, #0]// Causing a crash
LDUR X11, [X0, #0] // Okay
```

- Complaints about teammates will NOT be considered after the beginning of week 8.

6 Project Submission

- **The project is due before Thursday, June 4th lecture (Week 10) at 11:00 AM.**
- You must submit your solution on Canvas as a completed file named `ABC_XYZ_2026_project.s`, where ABC and XYZ are the email addresses, excluding @ucsd.edu, for each student. **Handwritten code and code in .pdf format will not be accepted.**
- Fill in the names and PID of each student at the top of the file.
- Each project team should contribute to their own unique code – no copying, cheating, hiring help, or any generative AI.